# Knowing When Not to Correct: Calibrated Uncertainty Gating for Vision-Language-Action Model Refinement

**Anonymous Author(s)**
Affiliation
Address
email

**Abstract:** We show that learned post-hoc correction of frozen flow-matching Vision-Language-Action (VLA) models is harmful by default—not because the correction module is poor, but because the vast majority of control steps do not need it. On most steps, the VLA already produces confident, low-variance actions; corrections applied here act as noise, degrading performance across all evaluated task suites and model scales. This motivates a shift from improving correction quality to deciding *when to correct at all*. We propose a calibrated gating mechanism that draws two stochastic action samples from the frozen VLA in a single batched forward pass—at zero extra latency—and uses their pairwise distance as a per-step uncertainty proxy. A threshold on this distance gates a lightweight correction module: confident steps pass through unchanged, while only genuinely uncertain steps receive learned residual correction. Across four LIBERO manipulation suites on two VLAs spanning an order of magnitude in scale, selective gating recovers nearly all degradation caused by unconditional correction, with recovery magnitude correlating monotonically with task difficulty. The key finding is that knowing when *not* to correct matters more than the correction itself.

**Keywords:** Vision-Language-Action Models, Inference-Time Refinement, Robotic Manipulation

## 1 Introduction

Frozen flow-matching Vision-Language-Action (VLA) models offer a compelling deployment paradigm: a single pretrained policy can handle diverse manipulation tasks without retraining [1, 2, 3]. Yet their action quality is bounded by the pretrained weights, and fine-tuning remains expensive when new tasks arrive frequently. Post-hoc learned correction—a lightweight module that refines actions at inference time without modifying VLA weights [4]—is an appealing alternative. But does it actually help?

We systematically evaluate this idea and find a surprising negative result: unconditional learned correction is universally harmful. Across four LIBERO manipulation suites [5] and two VLAs spanning an order of magnitude in scale, applying a correction module at every control step consistently degrades performance (Section 4.3). The problem is not the correction module itself, but the assumption that every step needs correction.

Recent work addresses parts of this problem space without confronting the core failure mode. A2C2 [6] learns a correction head on action chunks but applies it unconditionally and targets temporal staleness rather than initial action quality. ReconVLA [7] quantifies uncertainty over frozen VLAs but does not learn a corrective residual. SC-VLA [8] uses reinforcement learning and therefore abandons the weight-frozen constraint. VLA-Pilot [9] performs evolutionary search at 2.41 s per control step, too slow for closed-loop deployment. None analyze when correction helps versus hurts, nor provide a mechanism to suppress correction when the VLA is already confident.

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

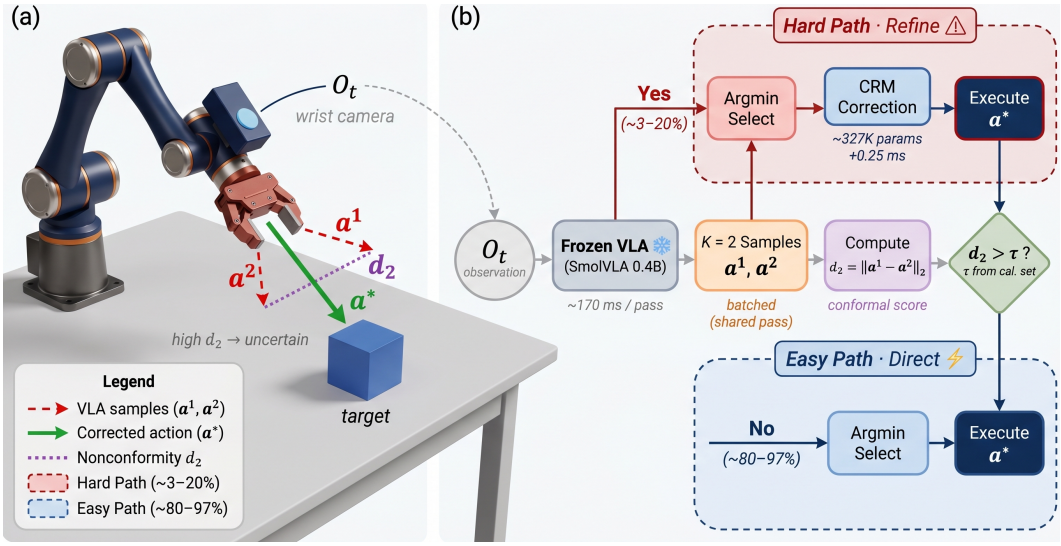



Figure 1: Calibrated gating pipeline. $K$=2 action samples are drawn from the frozen VLA; their pairwise $L_2$ distance forms a calibrated uncertainty score. When the score falls below $\tau$ (confident step), the argmin-to-centroid sample is output directly. When it exceeds $\tau$ (uncertain step), a lightweight CRM applies bounded correction ($\|\boldsymbol{\delta}\|_2 \leq \varepsilon$).

Our analysis reveals the root cause: the vast majority of control steps already produce confident, low-variance actions, and corrections on these steps inject noise rather than signal. This has a direct consequence: knowing *when not to correct* matters more than the correction's magnitude. We propose a simple gating mechanism: two stochastic action samples drawn in a single batched forward pass provide a per-step uncertainty estimate at zero extra latency. A calibrated threshold on their pairwise distance gates correction to the small fraction of genuinely uncertain steps, while the VLA's own output passes through on confident steps. This selective prediction approach [10, 11] restricts correction to the minority of steps where it is most needed.

On SmolVLA [2] (0.4B) across four LIBERO suites, selective gating recovers nearly all degradation from unconditional correction. Cross-VLA experiments on $\pi_{0.5}$ [12] (3.6B) provide preliminary evidence that the failure pattern persists and amplifies on stronger models. Threshold sensitivity analysis shows that gating is robust to $\tau$ choice, and trigger rate correlates monotonically with effect size across suites (Appendix C).

Our contributions are:

1. **Failure analysis of unconditional correction** (Section 4.3): we show that applying learned correction at every step degrades frozen VLAs across all evaluated task suites and model scales, and identify confident steps—where the VLA needs no help—as the source of harm.
2. **Calibrated gating via pairwise uncertainty** (Section 3): two action samples yield a per-step uncertainty proxy that gates correction to the small fraction of uncertain steps, recovering nearly all degradation at zero extra latency.
3. **Cross-suite and cross-VLA validation** (Section 4; Appendix B): trigger rate and recovery show monotonic correlation across suites, and the degradation pattern generalizes from SmolVLA (0.4B) to $\pi_{0.5}$ (3.6B), providing evidence that the mechanism is not architecture-specific.

## 2 Related Work

**VLA robustness.** Vision-language-action models have progressed from tokenized prediction [13, 14, 15] to generative approaches based on diffusion [16] and flow matching [1, 2]. Robustness evaluations [17] show that current VLAs lose over 65 pp under deployment perturbations. Training-

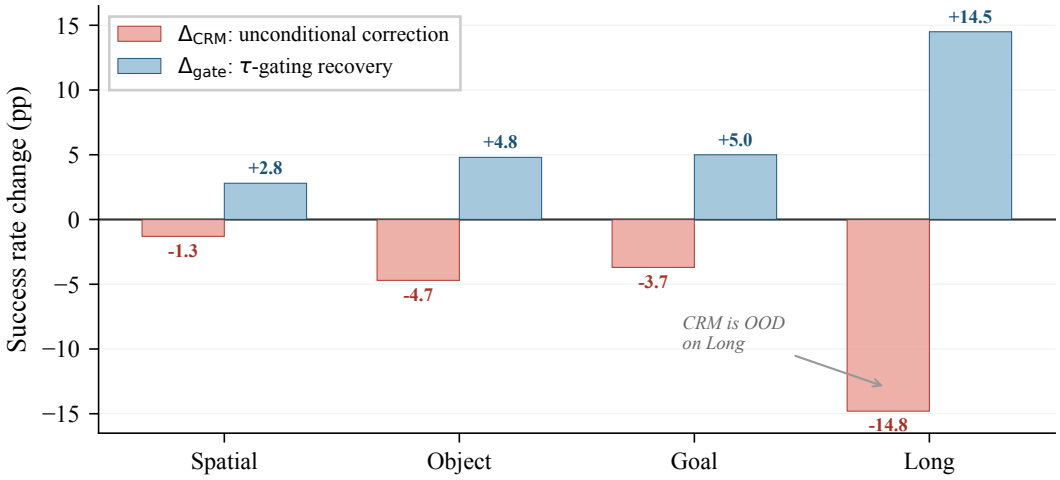



Figure 2: Degradation from unconditional correction ($\Delta_{\text{CRM}}$, red) vs. recovery from calibrated $\tau$-gating ($\Delta_{\text{gate}}$, blue) across four LIBERO suites on SmolVLA. Unconditional CRM correction degrades all suites (up to $-14.8$ pp on Long, where the CRM is out-of-distribution); $\tau$-gating recovers nearly all degradation by restricting correction to high-uncertainty steps.

time defenses (SRPO [18], RobustVLA [19], DiG-Flow [20], TT-VLA [21]) improve robustness but require retraining or reward engineering.

**Inference-time correction.** Among frozen-VLA methods, ReconVLA [7] provides uncertainty quantification but no learned correction; SC-VLA [8] requires task-level RL; VLA-Pilot [9] uses evolutionary optimization at 2.41 s per step. A2C2 [6] is the closest concurrent work: it learns a correction head addressing *temporal staleness* within action chunks. Our pipeline targets a complementary problem, namely detecting *when learned correction would hurt*, using calibrated gating to suppress a bounded correction module on confident steps. Residual policy learning [22, 23] shares the additive-correction structure; magnitude control is the central challenge, addressed by gating [24], RL-bounded clipping [25], or decay [26]. Our CRM uses fixed $L_2$ norm clipping with supervised loss, requiring no RL or online rollouts.

**Selective prediction and abstention.** Our $\tau$-gating mechanism is an instance of selective prediction [10, 27, 11], where the system abstains from applying correction when confidence is high. SelectiveNet [28] learns a joint selection-prediction network; we use a simpler pairwise distance threshold requiring no additional learned components. The $K{=}2$ pairwise $L_2$ distance is a minimal form of ensemble disagreement [29], related to MC-Dropout [30] but requiring no architectural modification. In robotics, confidence-gated autonomy [31] uses learned confidence for human-robot handoff; our gating switches between corrected and uncorrected actions within a fully autonomous pipeline. Conformal prediction [32] has been applied to robotics for safe planning [33], teleoperation [34], and crowd navigation [35]; our calibrated threshold draws on this calibration philosophy but does not implement formal coverage guarantees.

**Inference-time compute scaling.** Test-time scaling in LLMs [4] improves quality via best-of-N selection. RoVer [36] transfers this to VLAs with a reward model; DA-SIP [37] adapts denoising steps per difficulty. Our empirical analysis shows a different pattern in continuous action spaces: learned correction is universally harmful when applied unconditionally, and the key mechanism is calibrated gating, i.e., knowing *when not to correct*, rather than scaling sample count (Section 4.2).

## 3 Method: Calibrated Uncertainty Gating for Learned Correction

We propose a calibrated gating pipeline for frozen flow-matching VLAs (Figure 1). The calibration is based on held-out percentile thresholds of pairwise distances; it does not implement formal

conformal prediction guarantees [32]. Since unconditional learned correction degrades all evaluated suites (Section 4.3), the pipeline uses a $K{=}2$ pairwise distance to decide per step whether correction is safe. At each control step, the pairwise $L_2$ distance between two stochastic action samples forms an uncertainty score; confident steps output the selected sample directly, while uncertain steps apply a learned residual correction via a lightweight Correction Residual Module (CRM). The base VLA is never retrained.

### 3.1 Problem Setup

Let $\pi_\theta$ be a frozen flow-matching VLA that, given an observation $o_t$ (image, language instruction, proprioceptive state), generates action chunks of length $C{=}50$ in $\mathbb{R}^{d\times C}$ with $d{=}6$ (3 translation + 3 rotation). Under receding-horizon execution, only the first $H{=}5$ steps of each chunk influence the robot before re-planning, so we discard the remainder. The gripper dimension is handled separately via direct thresholding and is excluded from the pipeline.

Each forward pass of $\pi_\theta$ produces a stochastic action sample; querying independently $K$ times yields $K$ candidate chunks. Our goal is to produce a single action more accurate than any individual sample, with no changes to $\pi_\theta$. The method applies to any flow-matching VLA that produces diverse samples (Appendix A).

### 3.2 Multi-Sample Selection

We draw $K$ action chunks from $\pi_\theta$ via independent flow-matching rollouts and truncate each to $H{=}5$ steps, yielding samples $\{a_h^{(k)}\}_{k=1}^K$ with $a_h^{(k)} \in \mathbb{R}^d$ for horizon step $h = 1, \ldots, H$. For each step, we compute the sample mean $\boldsymbol{\mu}_h = \frac{1}{K}\sum_{k=1}^K a_h^{(k)}$ and select the sample with the smallest total $L_2$ distance to the mean across all horizon steps:

$$a_{\text{base}} = \underset{k\in\{1,\ldots,K\}}{\arg\min} \sum_{h=1}^{H} \left\| a_h^{(k)} - \boldsymbol{\mu}_h \right\|_2^2. \tag{1}$$

Unlike the element-wise mean, which can blend incompatible modes, the argmin returns an actual sample and preserves within-sample temporal coherence. At $K{=}2$, the argmin-to-centroid cost is identical for both samples ($\|a^{(1)} - \boldsymbol{\mu}\| = \|a^{(2)} - \boldsymbol{\mu}\| = \|a^{(1)} - a^{(2)}\|/2$), so selection reduces to a deterministic index tie-break; the value of $K{=}2$ lies not in selection quality but in the pairwise distance signal for gating (Section 3.5).

### 3.3 Correction Module (CRM)

The CRM is a lightweight MLP that refines the selected action $a_{\text{base}}$ by predicting a bounded residual correction conditioned on the current observation and the selected action itself.

**Architecture.** The CRM consists of three fully connected layers with ReLU activations: $\text{Linear}(n_{\text{in}}, 256) \to \text{ReLU} \to \text{Linear}(256, 256) \to \text{ReLU} \to \text{Linear}(256, H{\cdot}d)$, preceded by input normalization. The input is the concatenation of the frozen observation embedding $\mathbf{z}_{\text{obs}} \in \mathbb{R}^{n_{\text{obs}}}$ from the VLA's vision-language encoder and the selected action $a_{\text{base}} \in \mathbb{R}^{H\cdot d}$ ($H{=}5$ steps $\times$ $d{=}6$ DOFs), giving $n_{\text{in}} = n_{\text{obs}} + 30$. The architecture is identical across VLAs; only $n_{\text{obs}}$ differs (960 for SmolVLA, 2048 for $\pi_{0.5}$), yielding 327K–615K parameters ($< 1\%$ of either backbone). The output is a residual $\boldsymbol{\delta} \in \mathbb{R}^{30}$, and the corrected action is:

$$\mathbf{a}^* = a_{\text{base}} + \text{clip}\big(\boldsymbol{\delta},\ \|\boldsymbol{\delta}\|_2 \le \varepsilon\big), \tag{2}$$

where $\varepsilon$ is a norm bound that limits the correction magnitude (Section 3.4). The CRM never sees individual action candidates: it receives only the already-selected $a_{\text{base}}$ and predicts a correction toward the expert action. We tested alternative conditioning inputs (per-DOF uncertainty features, scalar uncertainty); observation-only conditioning performs best and is the primary variant.

Table 1: Uncertainty proxy comparison. Spearman $\rho$ between candidate proxies and $K{=}10$ action variance (ground-truth uncertainty), evaluated on LIBERO-Object (27,626 steps). Only pairwise $L_2$ distance achieves meaningful correlation.

| Proxy | Spearman $\rho$ | Extra VLA calls |
|---|---|---|
| $K{=}2$ pairwise $L_2$ (ours) | $\mathbf{0.9979}$ | 1 |
| $K{=}1$ CRM correction magnitude | 0.152 | 0 |
| $K{=}2$ CRM correction magnitude | 0.134 | 1 |
| Flow velocity dispersion (FMVD) | 0.083 | 0 |

**Training.** The CRM is trained offline on demonstration trajectories from LIBERO [5] with $L_2$ loss:

$$\mathcal{L} = \left\|\mathbf{a}^* - \mathbf{a}_{\text{expert}}\right\|_2^2, \tag{3}$$

where $\mathbf{a}_{\text{expert}}$ is the demonstration action. For each VLA, we precompute training data from 10 demonstrations per task across the target LIBERO suites. At each demonstration step, we run $K{=}10$ forward passes through the frozen VLA, apply argmin selection (1) to obtain $a_{\text{base}}$, and train the CRM to predict the residual toward $\mathbf{a}_{\text{expert}}$. Training uses Adam (lr$=10^{-3}$), batch size 32, 50 epochs, with Gaussian noise augmentation on $a_{\text{base}}$ (static perturbation, not interactive aggregation as in DAgger [38]) to improve generalization to the closed-loop state distribution. Note that the CRM is trained on $a_{\text{base}}$ obtained via $K{=}10$ argmin (well-centered sample), whereas at deployment with $K{=}2$ the argmin is degenerate (Section 3.2), creating a train–test distribution shift that may partially explain CRM underperformance.

### 3.4 Magnitude Control

The norm bound $\varepsilon$ in (2) is critical for stable closed-loop performance. Without clipping, the CRM's unconstrained output norm ($\approx$1.42) causes compounding errors. Constraining $\varepsilon$ to $[0.05, 0.2]$ yields stable corrections (Appendix D). Both CRM always-on and CRM $\tau$-gated use $\varepsilon{=}0.2$; the K-matched design ensures the only difference is gating (Section 4.3).

### 3.5 Calibrated Uncertainty Gating

Experiments reveal that unconditional correction is universally harmful: CRM applied at every step degrades all four LIBERO suites (up to $-14.8$ pp; Section 4.3). The 80–97% of steps where the VLA is already confident receive corrections that are noise. The solution is a calibrated gating mechanism that detects confident steps and suppresses correction accordingly.

**Why $K{=}2$.** $K{=}2$ is the minimum sample size that admits a pairwise disagreement measure, and under batched inference the uncertainty signal is *free*: $K{=}2$ latency ($\sim$170 ms) matches $K{=}1$ because both samples share the same prefix encoding and flow-matching denoising runs in parallel (Table 3). Empirically, the $K{=}2$ $L_2$ distance ranks steps in nearly identical order to $K{=}10$ sample variance ($\rho{=}0.9979$; Table 1), establishing ordering consistency of the proxy; this measures relative uncertainty ranking, not predictive validity for action correctness. Step-level validity is established indirectly through the suite-level monotonic correlation between trigger rate and performance recovery (Figure 4). Three alternative probes, including CRM correction magnitude ($K{=}1$ and $K{=}2$) and flow-matching velocity dispersion (FMVD), all fail ($\rho < 0.16$). Under the K-matched comparison ($K{=}2$ for both), calibrated $\tau$-gating exceeds unconditional correction on all four evaluated suites ($\Delta_{\text{gate}}{=}+2.8$ to $+14.5$ pp; Section 4.2).

**Three-step gating pipeline.** At each control step, the pipeline executes:

1. **Uncertainty probe.** Draw $K{=}2$ samples $a^{(1)}, a^{(2)}$ from $\pi_\theta$ in a single batched forward pass. Compute the pairwise distance $d_2 = \|a^{(1)} - a^{(2)}\|_2$ and select $\hat{a}$ via (1); at $K{=}2$, both candidates have equal cost to the centroid, so a deterministic index is used.
2. **Gating decision.** If $d_2 \le \tau$ (low uncertainty), output $\hat{a}$ directly, requiring no further computation. If $d_2 > \tau$ (high uncertainty), pass $\hat{a}$ to the CRM for correction.

3. **Learned correction.** For high-uncertainty steps only, the CRM predicts a bounded residual: $\hat{a}_{\text{out}} = \hat{a} + \boldsymbol{\delta}$, where $\boldsymbol{\delta} = \text{CRM}(o_t, \hat{a})$ and $\|\boldsymbol{\delta}\|_2 \leq \varepsilon$ (2). The pipeline uses at most $K{=}2$ forward passes; performance gains come from learned correction, not brute-force sampling.

The threshold $\tau$ is a percentile of the empirical $d_2$ distribution from CRM training data; it controls a precision–recall tradeoff: lower $\tau$ corrects more steps (higher recall, but risks correcting confident steps), while higher $\tau$ corrects fewer steps (higher precision, preserving baseline on confident steps). The optimal percentile is task-difficulty-dependent (Appendix C). CRM inference adds $\sim$0.25 ms; the VLA forward pass ($\sim$170 ms batched) dominates latency.

# 4 Experiments

We investigate when test-time correction helps or hurts on SmolVLA [2] (0.4B) across four LIBERO [5] task suites. We ask: (i) does unconditional correction degrade performance, and if so, why? (ii) can calibrated gating recover the degradation? (iii) what is the computational cost?

## 4.1 Setup

**Benchmarks.** We use all four LIBERO suites: Spatial (10 tasks, spatial reasoning), Object (10 tasks, novel object instances), Goal (10 tasks, goal specification), and Long (10 tasks, long-horizon manipulation). Each task is evaluated over 20 rollouts (max 300 steps, 200 per suite); success is binary.

**Base VLA.** SmolVLA [2] (0.4B parameters, flow matching) uses dual-camera RGB ($256{\times}256$), 8D proprioceptive state, and produces 7D actions (6D pose + 1D gripper, thresholded separately). We additionally report $\pi_{0.5}$ [12] (3.6B) baselines and OpenVLA-OFT [14] (7B) boundary analysis in Appendix B.

**Methods.** All methods use the same frozen SmolVLA; no VLA weights are modified. **VLA-only** ($K{=}1$) uses a single forward pass with no correction. **CRM always-on** ($K{=}2$, $\varepsilon{=}0.2$, $\tau{\to}0$) applies argmin selection and CRM correction at every step; it shares the same $K$, $\varepsilon$, and CRM checkpoint as CRM $\tau$-gated, with calibrated gating disabled. **CRM $\tau$-gated** ($K{=}2$, $\varepsilon{=}0.2$, $\tau{=}0.25$) is the proposed calibrated gating pipeline (Section 3.5), applying CRM correction only when $d_2 > \tau$. Additional ablations ($K{=}10$ argmin [7], $K{=}2$ mean) are reported in Appendix I.

**CRM training.** The CRM ($\sim$327K parameters) is trained on LIBERO demonstrations with 10 demos per task, $K{=}10$ forward passes each (Adam $\text{lr}{=}10^{-3}$, batch 32, 50 epochs, $\varepsilon{=}0.2$). Both CRM always-on and CRM $\tau$-gated use the same CRM checkpoint and $\varepsilon$. The CRM is trained on three suites (Spatial, Object, Goal; 30 tasks, $\sim$88K samples). Long is *not* included in CRM training; Long-suite evaluation therefore tests cross-suite generalization of the CRM (Appendix F).

**Statistics.** We report mean $\pm$ std over $n$ seeds ($n{=}3$ unless noted). An extended 8-seed analysis on Spatial yields $p{=}0.017$ (Appendix H). The primary comparison is CRM $\tau$-gated vs. CRM always-on under the K-matched design.

## 4.2 Main Results

Table 2 and Figure 3 report task success rates. Unconditional correction degrades all four suites: $-14.8$ pp on Long ($58.0{\to}43.2\%$), $-4.7$ pp on Object, $-3.7$ pp on Goal, $-1.3$ pp on Spatial. The CRM applies a learned residual at every step, including the 80–97% where the VLA is already confident; on these steps, corrections are noise. Calibrated $\tau$-gating recovers nearly all degradation: $+1.5$ pp on Spatial (trigger $\sim$20%), $+1.3$ pp on Goal (trigger $\sim$8%), and baseline-preserving on Object ($+0.1$ pp, trigger $\sim$4%) and Long ($-0.3$ pp, trigger $\sim$3%). Long—where the CRM is out-of-distribution—provides the clearest signal: unconditional correction loses 14.8 pp; calibrated

Table 2: Task success rate (% ↑) on SmolVLA across four LIBERO suites. CRM always-on and CRM $\tau$-gated share identical $K{=}2$, $\varepsilon{=}0.2$, and CRM checkpoint; the only difference is whether correction is applied unconditionally ($\tau{\to}0$) or selectively (calibrated $\tau$). $\Delta_{\text{CRM}}$: degradation from unconditional correction (CRM$-$VLA). $\Delta_{\text{gate}}$: recovery from gating ($\tau$-gated$-$CRM). Trigger: fraction of steps where the calibrated gate fires.

| Suite | VLA-only ($K{=}1$) | CRM always-on ($K{=}2$) | CRM $\tau$-gated ($K{=}2$) | $\Delta_{\text{CRM}}$ | $\Delta_{\text{gate}}$ | Trigger (%) |
|---|---|---|---|---|---|---|
| Spatial | $79.3 \pm 0.8$ | $78.0 \pm 1.7$ | $\mathbf{80.8 \pm 1.1}$ | $-1.3$ | $+2.8$ | $\sim$20 |
| Object | $77.7 \pm 1.3$ | $73.0 \pm 1.8$ | $\mathbf{77.8 \pm 2.1}$ | $-4.7$ | $+4.8$ | $\sim$4 |
| Goal | $83.7 \pm 1.3$ | $80.0 \pm 0.5$ | $\mathbf{85.0 \pm 1.3}$ | $-3.7$ | $+5.0$ | $\sim$8 |
| Long | $\mathbf{58.0 \pm 1.5}$ | $43.2 \pm 2.3$ | $57.7 \pm 1.5$ | $-14.8$ | $+14.5$ | $\sim$3 |

$n{=}3$ seeds. Bold: best per suite. $\tau$-gating recovers nearly all degradation; Long residual ($-0.3$ pp) within noise.

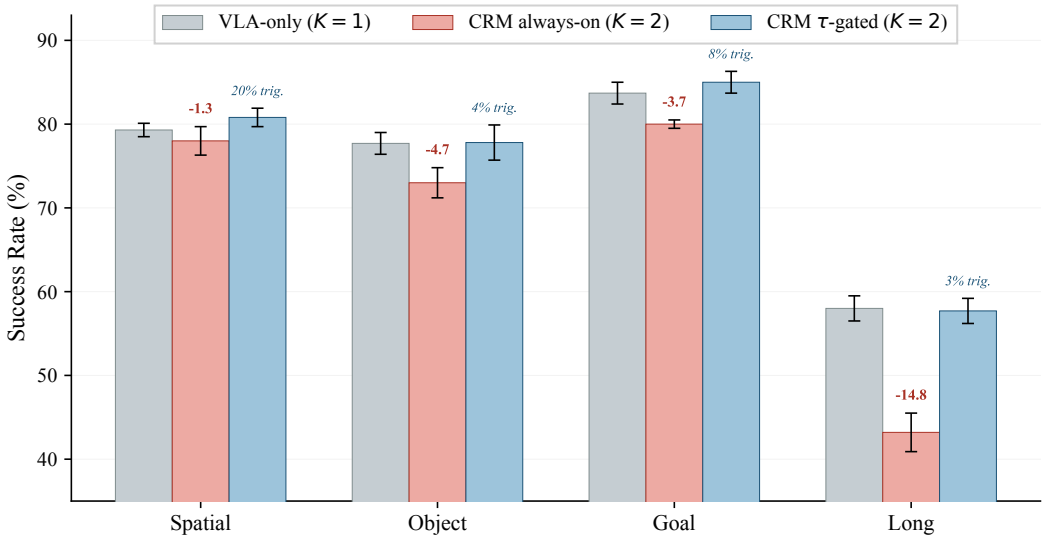



Figure 3: Success rates across four LIBERO suites under the K-matched comparison. CRM always-on ($K{=}2$, orange) degrades all four suites relative to VLA-only; CRM $\tau$-gated ($K{=}2$, blue) recovers nearly all degradation by selectively gating correction via calibrated confidence sets. Error bars: $\pm 1$ std over $n{=}3$ seeds.

gating at $\sim$3% trigger rate recovers 14.5 pp ($\Delta_{\text{gate}}{=}{+}14.5$). An 8-seed analysis on Spatial yields $p{=}0.017$ (Appendix H).

### 4.3 Correlation Between Trigger Rate and Recovery

Figure 4 visualizes the mechanism: trigger rate correlates monotonically with effect size. Suites with high uncertainty (trigger $>$7%: Spatial, Goal) see positive corrections; confident suites (trigger $<$5%: Object, Long) preserve baseline, avoiding the catastrophic $-14.8$ pp and $-4.7$ pp degradation of unconditional correction. SR is robust to $\tau$ choice, varying by $\leq$1 pp on Spatial as trigger rate changes $5.5\times$ (Appendix C).

**Efficiency.** Under batched inference with shared prefix encoding, $K{=}2$ latency matches $K{=}1$ ($\sim$170 ms), making the uncertainty probe computationally free. CRM adds $\sim$0.25 ms per call; $\tau$-gating invokes it on only 3–20% of steps (Table 3).

Cross-VLA validation on $\pi_{0.5}$ [12] (3.6B) confirms the degradation pattern generalizes: unconditional CRM correction degrades all four suites ($-2.0$ to $-17.0$ pp; Appendix B, Table 4). A boundary analysis on OpenVLA-OFT [14] (7B, deterministic) shows that stochastic diversity is a prerequisite for multi-sample calibrated scoring (Appendix A).

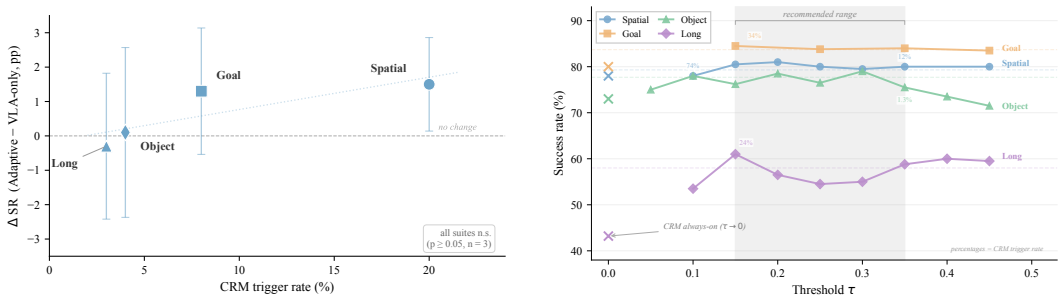



Figure 4: **Left:** Trigger rate vs. improvement over VLA-only ($\Delta_{\tau\text{-gated}}$). Suites where the gate fires on $>7\%$ of steps (Spatial, Goal) see positive improvement; confident suites (Object, Long, trigger $<5\%$) preserve baseline. **Right:** Threshold sensitivity on SmolVLA. Solid: CRM $\tau$-gated SR; dashed: VLA-only baselines; $\times$: CRM always-on. Gating is robust across a wide range of $\tau$.

Table 3: Computational comparison. CRM $\tau$-gated and CRM always-on share the same $K{=}2$ VLA cost; the difference is CRM invocation frequency. CRM overhead per call is negligible ($\sim$0.25 ms).

| Method | VLA calls | Latency (ms) | CRM calls (%) |
|---|---|---|---|
| VLA-only | 1 | $\sim$170 | 0 |
| CRM always-on | 2 | $\sim$170 | 100 |
| **CRM $\tau$-gated** | **2** | **$\sim$170** | **3–20** |

## 5 Conclusion

We showed that learned post-hoc correction of frozen flow-matching VLAs is harmful by default. Under a K-matched controlled comparison, unconditional correction degrades all four LIBERO suites on SmolVLA (0.4B) and all four suites on $\pi_{0.5}$ (3.6B; preliminary single-seed), with the largest degradation on Long where the CRM is out-of-distribution. The root cause is that the vast majority of control steps are already confident; corrections on these steps inject noise. Calibrated confidence sets from $K{=}2$ pairwise $L_2$ distances detect confident steps and gate the CRM, recovering nearly all degradation. This is a diagnostic contribution: calibrated gating makes learned correction modules safe to deploy; without one, VLA-only remains the simpler choice. The key finding is that *knowing when not to correct* matters more than correction strength.

## 6 Limitations

Paired improvements ($\tau$-gated vs. VLA-only) reach statistical significance on Spatial with extended seeds (Appendix H) but do not reach significance on other suites at the default seed count. Our finding is established for the tested lightweight MLP-based CRM; whether larger correction architectures or online fine-tuning would change this conclusion remains open. The threshold $\tau$ is suite-dependent (Appendix C), recovery on $\pi_{0.5}$ is partial (Appendix, Table 6), and all experiments use LIBERO simulation; adaptive $\tau$ selection, real-robot transfer, and broader cross-architecture validation remain future work.

## References

[1] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. $\pi_0$: A vision-language-action flow model for general robot control, 2024.

[2] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene. Smolvla: A vision-language-action model for affordable and efficient robotics, 2025.

[3] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

[4] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling llm test-time compute optimally can be more effective than scaling parameters for reasoning. In *International Conference on Learning Representations (ICLR)*, 2025.

[5] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[6] K. Sendai, M. Alvarez, T. Matsushima, Y. Matsuo, and Y. Iwasawa. Leave no observation behind: Real-time correction for vla action chunks, 2025.

[7] L. Chen, Z. Lyu, and W. J. Beksi. Reconvla: An uncertainty-guided and failure-aware vision-language-action framework for robotic control, 2026.

[8] C. Liu, W. Tan, L. Zhu, F. Li, J. Li, G. Yang, and H. T. Shen. Self-correcting vla: Online action refinement via sparse world imagination, 2026.

[9] Z. Li, J. Liu, Z. Dong, T. Teng, Q. Rouxel, D. Caldwell, and F. Chen. Towards deploying vla without fine-tuning: Plug-and-play inference-time vla policy steering via embodied evolutionary diffusion, 2025.

[10] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970.

[11] Y. Geifman and R. El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[12] P. Intelligence, K. Black, N. Brown, J. Bradbury, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, O. Mees, M. Mothukuri, S. Nair, K. Pertsch, A. Ren, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, and H. Wang. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025.

[13] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control, 2023.

[14] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. Openvla: An open-source vision-language-action model, 2024.

[15] D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy, 2024.

[16] C. Chi, Z. Xu, S. Feng, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023. Also published in IJRR 2024. arXiv:2303.04137.

[17] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu. Libero-plus: In-depth robustness analysis of vision-language-action models, 2025.

[18] S. Fei, S. Wang, L. Ji, A. Li, S. Zhang, L. Liu, J. Hou, J. Gong, X. Zhao, and X. Qiu. Srpo: Self-referential policy optimization for vision-language-action models, 2025.

[19] H. Zhang, S. Zhang, J. Jin, Q. Zeng, R. Li, and D. Wang. Robustvla: Robustness-aware reinforcement post-training for vision-language-action models, 2025.

[20] W. Zhang, Y. Wang, H. Luo, H. Yuan, Y. Feng, S. Zheng, Q. Jin, and Z. Lu. Dig-flow: Discrepancy-guided flow matching for robust vla models, 2025.

[21] C. Liu, Y. Liu, T. Wang, Q. Zhuang, J. C. Liang, W. Yang, R. Xu, Q. Wang, D. Liu, and C. Han. On-the-fly vla adaptation via test-time reinforcement learning, 2026.

[22] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling. Residual policy learning. In *Robotics: Science and Systems (RSS)*, 2018. arXiv:1812.06298.

[23] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine. Residual reinforcement learning for robot control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 6023–6029, 2019.

[24] Y. Jiang, C. Wang, R. Zhang, J. Wu, and L. Fei-Fei. Transic: Sim-to-real policy transfer by learning from online correction. In *Conference on Robot Learning (CoRL)*, 2024. arXiv:2405.10315.

[25] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan, G. Shi, and Y. Zhu. Self-improving vision-language-action models with data generation via residual rl. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2511.00091.

[26] C. Pan, H. H. Cheng, and J. Hughes. Online imitation learning for manipulation via decaying relative correction through teleoperation, 2025.

[27] R. El-Yaniv and Y. Wiener. On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11:1605–1646, 2010.

[28] Y. Geifman and R. El-Yaniv. Selectivenet: A deep neural network with an integrated reject option. In *International Conference on Machine Learning (ICML)*, 2019.

[29] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[30] Y. Gal and Z. Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *International Conference on Machine Learning (ICML)*, 2016.

[31] R. Hoque, L. Y. Chen, S. Sharma, K. Dharmarajan, and K. Goldberg. Confidence-gated supervised autonomy, 2025.

[32] A. N. Angelopoulos and S. Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *Foundations and Trends in Machine Learning*, 2023.

[33] L. Lindemann, M. Cleaveland, G. Shim, and G. J. Pappas. Safe planning in dynamic environments using conformal prediction. *IEEE Robotics and Automation Letters*, 8(8):5116–5123, 2023.

[34] J. Luo et al. Sample-efficient confidence-based shared autonomy via conformal prediction. In *Robotics: Science and Systems (RSS)*, 2024.

[35] J. Chen et al. Conformal prediction for safe crowd navigation. In *Conference on Robot Learning (CoRL)*, 2025.

[36] Y. Dai, H. Liu, Z. Bai, Y. Liu, J. Wang, H. Su, C. Chen, J. Lin, and L. Wu. Rover: Robot reward model as test-time verifier for vision-language-action model, 2025.

[37] S. Chun, J. Lee, M. S. Albergo, L. Xie, and E. Vanden-Eijnden. Dynamic test-time compute scaling in control policy: Difficulty-aware stochastic interpolant policy. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[38] S. Ross, G. J. Gordon, and J. A. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

# A Applicability Boundary: Stochastic VLA Diversity

The multi-sample pipeline requires stochastic action diversity across samples. We validate this prerequisite on two VLA architectures: SmolVLA [2] (flow matching, 0.4B) and OpenVLA-OFT [14] (deterministic $L_1$ regression, 7B).

SmolVLA produces meaningfully diverse action samples across all dimensions via its flow-matching generative process. In contrast, OpenVLA-OFT's deterministic $L_1$ regression produces near-identical outputs: mean pairwise $L_2$=0.00436 across $K$=2 samples, with 97.3% of steps below 0.01. The uncertainty gate never fires, and multi-sample selection is vacuous.

This result establishes an architectural boundary: multi-sample inference-time correction is specific to stochastic generative VLAs (flow matching, diffusion). Deterministic VLAs lack the action diversity that selection and correction rely on. The boundary is architectural, not a limitation of our method: any inference-time approach that draws multiple action candidates requires the base policy to produce diverse samples.

# B Cross-VLA Analysis: Model Capability Dependency

We evaluate the CRM degradation pattern on $\pi_{0.5}$ [12] (3.6B, flow matching), where near-saturated baselines reach 94–98% (Table 4). The same K-matched design ($K$=2, $\varepsilon$=0.2, shared CRM architecture) is used. Unconditional CRM correction degrades all four suites: Long suffers −17.0 pp (94.0→77.0%, the largest single-suite drop across both VLAs), Goal −14.0 pp, Object −6.0 pp, and Spatial −2.0 pp. As on SmolVLA, the CRM is trained on three suites and Long is out-of-distribution, amplifying the harm. The degradation on all four suites of both VLAs is consistent with the core thesis: when most steps are already near-optimal, learned correction noise has proportionally greater impact, and this pattern generalizes across model scale.

Table 4: Cross-VLA degradation analysis: $\pi_{0.5}$ (3.6B) on LIBERO. Near-saturated baselines (94–98%) are degraded by unconditional CRM correction on all four suites, with Long suffering −17.0 pp (the largest drop across both VLAs).

| Suite | VLA-only ($K$=1) | CRM always-on ($K$=2) | $\Delta_{\text{CRM}}$ |
|---|---|---|---|
| Spatial | 98.0 | 96.0 | −2.0 |
| Object | 96.5 | 90.5 | −6.0 |
| Goal | 95.5 | 81.5 | −14.0 |
| Long | 94.0 | 77.0 | −17.0 |

$n$=1 seed; $\varepsilon$=0.2, same CRM architecture as SmolVLA.
VLA-only and CRM always-on evaluated on different servers (seed-matched); cross-server variance is ≤2.5 pp on short suites, and all degradation directions are consistent.

# C Threshold Sensitivity Analysis

Tables 5 and 6 report success rates as a function of the uncertainty threshold $\tau$ on SmolVLA and $\pi_{0.5}$, respectively. On both models, $\tau$ primarily controls intervention frequency rather than final SR: on SmolVLA Spatial, trigger rate drops from 40.7% ($\tau$=0.15) to 7.3% ($\tau$=0.45), a 5.5× reduction, while SR varies by only 0.5 pp (80.0–80.5%). On $\pi_{0.5}$ Spatial, a similar pattern holds (11.2%→2.6% trigger, 0.5 pp SR range). This plateau effect indicates that the CRM needs to intervene on only a small fraction of steps to reach its performance ceiling.

The Long suite, where the CRM is OOD, exhibits wider SR variation across $\tau$: from 53.5% ($\tau$=0.10, trigger 61.3%) to 61.0% ($\tau$=0.15, trigger 23.8%). Notably, $\tau$=0.15 exceeds VLA-only (58.0%) by 3 pp, while the shared threshold $\tau$=0.25 yields 54.5%. This suite-dependence of optimal $\tau$ motivates adaptive threshold selection as future work (Section 6). On $\pi_{0.5}$ Goal, extending $\tau$ from 0.15 to 0.90 confirms the plateau: SR remains within 87.0–92.0% as trigger rate drops from 17.4% to 0.4%.

Table 5: SmolVLA threshold sensitivity ($\varepsilon$=0.2), complete $\tau$-sweep across all four LIBERO suites. $\tau$ primarily controls intervention frequency: on Spatial ($\tau$=0.15–0.45), trigger rate changes 5.5× while SR varies by $\leq$1 pp. Long is OOD for the CRM (Section 4.1).

| Suite | $\tau$ | SR (%) | Trigger (%) | $n$ |
|---|---|---|---|---|
| Spatial | 0.10 | 78.0 | 74.4 | 1 |
| Spatial | 0.15 | 80.5 | 40.7 | 1 |
| Spatial | 0.20 | 81.0 | 26.7 | 1 |
| Spatial | 0.25 | 80.0 | 19.5 | 3 |
| Spatial | 0.30 | 79.5 | 16.8 | 1 |
| Spatial | 0.35 | 80.0 | 12.2 | 1 |
| Spatial | 0.45 | 80.0 | 7.3 | 1 |
| Goal | 0.15 | 84.5 | 34.4 | 2 |
| Goal | 0.25 | 83.8 | 8.1 | 3 |
| Goal | 0.35 | 84.0 | 2.3 | 2 |
| Goal | 0.45 | 83.5 | 0.9 | 1 |
| Object | 0.05 | 75.0 | — | 1 |
| Object | 0.10 | 78.0 | — | 1 |
| Object | 0.15 | 76.2 | 16.6 | 3 |
| Object | 0.20 | 78.5 | — | 1 |
| Object | 0.25 | 76.5 | 3.5 | 2 |
| Object | 0.30 | 79.0 | 2.3 | 1 |
| Object | 0.35 | 75.5 | 1.3 | 1 |
| Object | 0.40 | 73.5 | 0.9 | 1 |
| Object | 0.45 | 71.5 | 0.8 | 1 |
| Long$^\dagger$ | 0.10 | 53.5 | 61.3 | 1 |
| Long$^\dagger$ | 0.15 | 61.0 | 23.8 | 2 |
| Long$^\dagger$ | 0.20 | 56.5 | 8.9 | 2 |
| Long$^\dagger$ | 0.25 | 54.5 | 4.4 | 2 |
| Long$^\dagger$ | 0.30 | 55.0 | — | 2 |
| Long$^\dagger$ | 0.35 | 58.8 | 1.2 | 2 |
| Long$^\dagger$ | 0.40 | 60.0 | 0.4 | 1 |
| Long$^\dagger$ | 0.45 | 59.5 | 0.3 | 1 |

$n$: seeds averaged. $^\dagger$CRM is OOD on Long (trained on 3 suites).
—: trigger rate not recorded for this run.

Goal and Object show slightly larger SR variation ($\sim$1–3 pp) across $\tau$ values, but these differences fall within the per-seed noise range reported in Table 2 (std $\sim$1–2 pp). The within-suite relationship between trigger rate and SR is not monotonic on these suites, consistent with the CRM providing limited correction quality on individual steps where the VLA's errors are systematic rather than stochastic. The cross-suite monotonic pattern (higher trigger rate correlating with larger improvement, Figure 4) remains the primary mechanism evidence.

**Note on $\varepsilon$.** Both CRM always-on and CRM $\tau$-gated in Table 2 use $\varepsilon$=0.2, matching the $\tau$-sweep configuration. The K-matched design ensures the only variable is calibrated gating ($\tau \to 0$ vs. calibrated $\tau$).

# D Correction Magnitude Sensitivity

Figure 5 shows how the norm bound $\varepsilon$ affects closed-loop success rate. The observation-only CRM is robust across $\varepsilon \in [0.05, 0.2]$ on both suites, while the per-DOF variant (which conditions on per-DOF uncertainty features) collapses at high $\varepsilon$, reaching 0% at unconstrained $\varepsilon$ on Spatial. This motivates the observation-only architecture and the $\varepsilon$=0.2 setting used throughout.

Table 6: $\pi_{0.5}$ threshold sensitivity ($\varepsilon$=0.2). The plateau pattern holds on all suites: trigger rate varies substantially with $\tau$, but SR changes remain small. Extended $\tau$ range on Goal (0.15–0.90) confirms robustness.

| Suite | $\tau$ | SR (%) | Trigger (%) | $n$ |
|---|---|---|---|---|
| Spatial | 0.15 | 97.0 | 11.2 | 1 |
| Spatial | 0.25 | 96.8 | 4.4 | 2 |
| Spatial | 0.35 | 96.5 | 2.6 | 1 |
| Goal | 0.15 | 92.0 | 17.4 | 1 |
| Goal | 0.25 | 87.5 | 9.1 | 3 |
| Goal | 0.35 | 87.5 | 5.7 | 1 |
| Goal | 0.50 | 87.0 | 2.1 | 1 |
| Goal | 0.70 | 87.0 | 1.0 | 1 |
| Goal | 0.90 | 89.0 | 0.4 | 1 |
| Long | 0.15 | 84.0 | 11.4 | 1 |
| Long | 0.25 | 81.0 | 4.4 | 3 |
| Long | 0.35 | 81.0 | 2.4 | 1 |

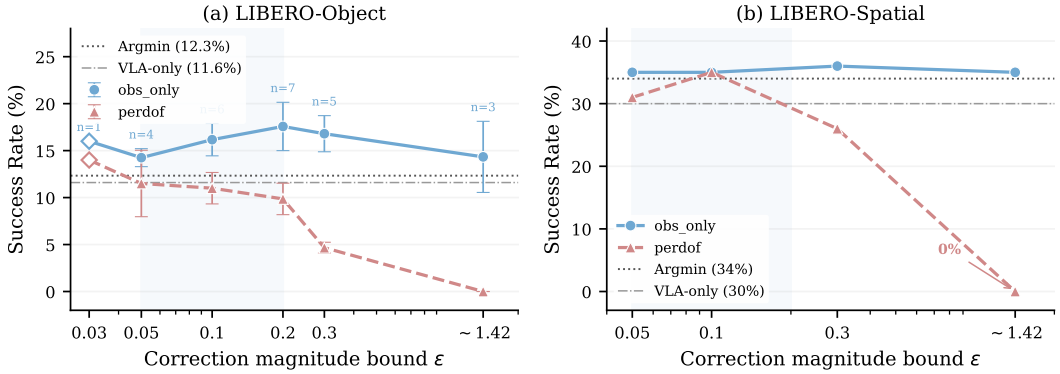



Figure 5: Correction magnitude sensitivity. (a) LIBERO-Object: the observation-only CRM (blue) is robust across $\varepsilon \in [0.05, 0.2]$, while the per-DOF variant (red) collapses at high $\varepsilon$. (b) LIBERO-Spatial (seed 42): same pattern, with per-DOF reaching 0% at unconstrained $\varepsilon$.

## E Uncertainty Proxy Comparison

Table 1 (main text) compares four candidate uncertainty proxies against $K$=10 sample variance as ground truth. CRM correction magnitude (the $L_2$ norm of the CRM's predicted residual) is attractive because it requires no extra VLA forward passes, but correlates weakly with true uncertainty ($\rho < 0.16$): the CRM predicts similar-magnitude corrections regardless of whether the VLA is confident or uncertain. FMVD, which measures dispersion in the flow-matching velocity field, performs worse still ($\rho$=0.083). The $K$=2 pairwise $L_2$ distance achieves near-perfect correlation ($\rho$=0.9979) at the cost of one additional VLA forward pass, which is absorbed by batched inference (Section G).

## F Reproducibility Details

**Hardware.** All experiments run on NVIDIA RTX 4090 GPUs (24 GB VRAM). SmolVLA inference uses a single GPU; CRM training and evaluation each use a single GPU.

**CRM training.** The CRM is trained on three suites (Spatial, Object, Goal; 30 tasks, ∼88K samples) for 50 epochs with Adam (lr=$10^{-3}$, batch size 32) in ∼6 GPU-hours on 1× RTX 4090. Long is excluded from CRM training; Long-suite evaluation tests cross-suite generalization. Precomputing the $K$=10 forward-pass cache (required once per suite) takes ∼2 GPU-hours per suite.

**Evaluation.** Each suite evaluation (20 rollouts $\times$ 10 tasks $\times$ 300 max steps) takes $\sim$2.5 GPU-hours. A full 5-method $\times$ 4-suite $\times$ 3-seed experiment requires $\sim$150 GPU-hours total.

**Software.** PyTorch 2.4, Transformers 4.45, LIBERO (MuJoCo-based). Code and CRM checkpoints will be released upon acceptance.

# G Computational Efficiency

Table 3 (main text) compares the computational cost of each method. The key insight is that batched inference absorbs the cost of the second VLA forward pass: $K{=}2$ latency matches $K{=}1$ because both samples share the same prefix encoding and flow-matching denoising runs in parallel.

# H Extended 8-Seed Analysis on Spatial

Table 7: Per-seed success rates (%) on LIBERO-Spatial for VLA-only vs. CRM $\tau$-gated ($K{=}2$, $\tau{=}0.25$). All 8 seeds use identical $K{=}2$ configuration (code-verified).

| Seed | VLA-only | CRM $\tau$-gated | $\Delta$ |
|---|---|---|---|
| 42 | 80.0 | 82.0 | $+2.0$ |
| 43 | 79.0 | 80.0 | $+1.0$ |
| 44 | 78.5 | 80.5 | $+2.0$ |
| 45 | 80.0 | 80.0 | $+0.0$ |
| 46 | 80.5 | 80.0 | $-0.5$ |
| 47 | 80.0 | 81.5 | $+1.5$ |
| 48 | 79.5 | 81.0 | $+1.5$ |
| 49 | 78.5 | 82.0 | $+3.5$ |
| Mean$\pm$std | $79.5 \pm 0.8$ | $80.9 \pm 0.9$ | $+1.4$ |

Table 7 reports per-seed results for the extended 8-seed paired analysis on LIBERO-Spatial. A paired $t$-test yields $\Delta{=}+1.4$ pp, $p{=}0.017$ ($n{=}8$), reaching statistical significance at $\alpha{=}0.05$. Six of eight seeds show positive $\Delta$; the two non-positive seeds ($\Delta{=}0.0$ and $\Delta{=}-0.5$) are consistent with measurement noise rather than systematic degradation. This analysis provides the strongest statistical evidence that calibrated $\tau$-gating produces a genuine improvement on the suite where the uncertainty gate fires most frequently (trigger rate $\sim$20%).

# I Additional Ablations

Table 8: Supplementary ablations on SmolVLA. $K{=}10$ argmin [7]: argmin-to-centroid selection over 10 samples, no learned correction. $K{=}2$ mean: centroid of two samples, no CRM. Neither consistently improves over VLA-only.

| Method | $K$ | Spatial (%) | Object (%) | Goal (%) | Long (%) |
|---|---|---|---|---|---|
| VLA-only | 1 | $79.3 \pm 0.8$ | $77.7 \pm 1.3$ | $83.7 \pm 1.3$ | $58.0 \pm 1.5$ |
| $K{=}10$ argmin | 10 | $80.3 \pm 1.0$ | $77.3 \pm 1.5$ | $80.8 \pm 0.3$ | $56.5^{\dagger}$ |
| $K{=}2$ mean | 2 | — | $77.5 \pm 2.3$ | $83.0 \pm 2.2$ | $57.0^{\dagger}$ |

$n{=}3$ seeds unless noted; $^{\dagger}n{=}1$; — = omitted (computational budget).

Table 8 reports two ablations that isolate multi-sample selection without learned correction. $K{=}10$ argmin invests $10\times$ the VLA forward passes but hurts on Goal ($-2.9$ pp) and Long ($-1.5$ pp): selection alone cannot fix systematic errors. $K{=}2$ mean offers no learned correction mechanism ($-0.7$ pp Goal, $-1.0$ pp Long; Long is single-seed preliminary), confirming that multi-sampling alone is insufficient.

## J CRM Always-On Per-Seed Analysis

Table 9: Per-seed success rates (%) for CRM always-on ($K$=2, $\varepsilon$=0.2, $\tau \to 0$). SmolVLA uses seeds 42–46 ($n$=5); Table 2 reports $n$=3 means from seeds 42–44. $\pi_{0.5}$ results are preliminary (limited seeds).

| Model | Suite | s42 | s43 | s44 | s45 | s46 | $n$ | Mean±Std |
|---|---|---|---|---|---|---|---|---|
| SmolVLA | Spatial | 79.0 | 76.0 | 79.0 | 77.0 | 78.5 | 5 | $77.9 \pm 1.3$ |
| SmolVLA | Object | 71.0 | 73.5 | 74.5 | 74.5 | 67.5 | 5 | $72.2 \pm 3.0$ |
| SmolVLA | Goal | 80.0 | 79.5 | 80.5 | 82.0 | 80.0 | 5 | $80.4 \pm 1.0$ |
| SmolVLA | Long | 41.0 | 45.5 | 43.0 | — | — | 3 | $43.2 \pm 2.3$ |
| $\pi_{0.5}$ | Spatial | 96.0 | — | — | — | — | 1 | 96.0 |
| $\pi_{0.5}$ | Object | 90.5 | 92.0 | — | — | — | 2 | $91.2 \pm 1.1$ |
| $\pi_{0.5}$ | Goal | 81.5 | — | — | — | — | 1 | 81.5 |
| $\pi_{0.5}$ | Long | 77.0 | — | — | — | — | 1 | 77.0 |

—: not evaluated. SmolVLA Long s45 incomplete (excluded).

Table 9 reports per-seed CRM always-on results. SmolVLA means over seeds 42–44 match Table 2; the extended 5-seed means show consistent degradation patterns across all suites. Object exhibits the highest per-seed variance (std=3.0), driven by seed 46 (67.5%), suggesting interaction between CRM noise injection and task-specific stochasticity.

## K $\pi_{0.5}$ Extended Multi-Seed Results

Table 10: $\pi_{0.5}$ (3.6B) per-seed success rates (%) across three methods and four LIBERO suites from dedicated multi-seed evaluation runs. Table 4 reports seed-42 matched pairs from separate runs; per-seed differences ($\leq$4 pp) reflect cross-server variance and evaluation stochasticity.

| Suite | Method | s42 | s43 | s44 | $n$ | Mean±Std |
|---|---|---|---|---|---|---|
| Spatial | VLA-only | 98.0 | 97.5 | 98.5 | 3 | $98.0 \pm 0.5$ |
| Spatial | CRM always-on | 96.5 | 97.0 | 97.0 | 3 | $96.8 \pm 0.3$ |
| Spatial | CRM $\tau$-gated | 96.5 | 98.0 | 97.0 | 3 | $97.2 \pm 0.8$ |
| Object | VLA-only | 100.0 | 98.5 | 99.5 | 3 | $99.3 \pm 0.8$ |
| Object | CRM always-on | 89.0 | 87.0 | 88.0 | 3 | $88.0 \pm 1.0$ |
| Object | CRM $\tau$-gated | 85.5 | 85.0 | 88.0 | 3 | $86.2 \pm 1.6$ |
| Goal | VLA-only | 98.0 | 97.0 | 97.5 | 3 | $97.5 \pm 0.5$ |
| Goal | CRM always-on | 85.5 | 83.0 | 85.0 | 3 | $84.5 \pm 1.3$ |
| Goal | CRM $\tau$-gated | 89.5 | 89.5 | — | 2 | $89.5 \pm 0.0$ |
| Long | VLA-only | 94.0 | 95.5 | 95.5 | 3 | $95.0 \pm 0.9$ |
| Long | CRM always-on | — | — | 76.0 | 1 | 76.0 |
| Long | CRM $\tau$-gated | 80.5 | 80.5 | 83.0 | 3 | $81.3 \pm 1.4$ |

—: not evaluated. VLA-only and CRM evaluated on different servers (seed-matched).

Table 10 extends the preliminary single-seed results in Table 4 to multi-seed evaluation on $\pi_{0.5}$. Unconditional CRM correction degrades all four suites relative to VLA-only, confirming the single-seed pattern. On Object, CRM $\tau$-gated (86.2%) performs below CRM always-on (88.0%), indicating that gating effectiveness depends on the quality of the underlying CRM corrections, which diminishes when the base policy is near-optimal (VLA-only 99.3%). On Long, $\tau$-gating recovers 5.3 pp relative to CRM always-on (81.3 vs. 76.0), consistent with OOD suppression. Goal shows partial recovery: $\tau$-gating (89.5) narrows the gap to VLA-only (97.5) relative to CRM always-on (84.5).

## L K-Sweep Analysis

Table 11 varies the sample count $K$ for CRM always-on on LIBERO-Object. Performance is relatively stable across $K$=2–20 (72.2–78.0%), with $K$=10 showing the highest point estimate but

Table 11: Effect of sample count $K$ on CRM always-on performance (SmolVLA, LIBERO-Object, $\varepsilon$=0.2). Increasing $K$ beyond 2 provides marginal improvement; $K$=10 yields the highest SR but at 5× the VLA cost.

| $K$ | VLA calls | Seeds | SR (Mean±Std) | $\Delta$ vs. $K$=2 |
|---|---|---|---|---|
| 2 | 2 | 5 | $72.2 \pm 3.0$ | — |
| 5 | 5 | 2 | $73.2 \pm 0.4$ | +1.0 |
| 10 | 10 | 1 | 78.0 | +5.8 |
| 20 | 20 | 2 | $74.5 \pm 1.4$ | +2.3 |

$K$=2: from Table 9 ($n$=5). $K$=10 uses $\varepsilon$=0.05 (single seed).

at 5× the VLA forward-pass cost and with only a single seed. The marginal gain from larger $K$ does not justify the computational cost, supporting the paper's use of $K$=2 with calibrated gating as the preferred operating point.

*AI tools assisted with code and manuscript.*